==========
Atari 2600 TIA Hardware Notes
==========

v1.0 6-March-2003
by Andrew Towers
mariofrog@bigpond.com

++++++++++
+++ TIA Hardware Notes (a Small Opus on the TIA)

Following is a whole bunch of notes on the TIA I made while I was trying to work out how the whole thing is put together. You'll need a copy of the TIA schematics to understand the more complicated bits of this since I was looking at them when I wrote it. According to my copy they were scanned in by Mark De Smet. They are now available for download from AtariAge at: http://www.atariage.com/2600/archives/schematics_tia/index.html

I started out searching through the stella archives for any info on triggering the players more than once per scanline (at the time I wanted to draw more than the 12 copies possible by flicker and 3-repeat) - and I came across Eckhard Stolberg's 'grid2' demo from Oct 1998, followed by a long series of threads over several months discussing how the technique actually manages to work =) From all the articles it looked like a complete black art and no-one had a theory that would explain it fully.

Then I came across the TIA-1A schematics, and proceeded to spend the next 3-4 days solid drinking copious amounts of coffee and taking very little sleep while I tried to figure the whole mess out from the gate level up. (hey, the 2600 is a new hobby, I can splurge ;) In the end I found that as usual writing a 'few quick notes' turned into 'writing a tutorial' or, 'a small opus on the TIA'. So, here we go.

++++++++++
+++ Polynomial Counters, what the heck is this?

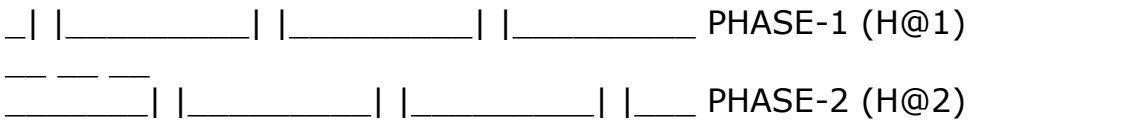
Almost all of the timing and counting within the TIA is implemented in the form of "polynomial counters", so this seems a good place to start. If you've never come across these before (I hadn't) they seem a really obscure way to go about counting things, but they're very small and simple to implement and therefore cheap on silicon. They also have the useful property that 'adding 1' takes linear time (unlike a ripple-carry adder/counter) - as long as you don't want to know where you're up to in traditional binary numeric form, they're perfect ;)

Actually, as the TIA designers pointed out, you can use a small lookup table to convert from one to the other, and you can compare two counter states to see if you're up to the same count without knowing the numeric values. But this is getting off track and hand-wavery. If you want to know the maths behind polynomial counters I suggest you look elsewhere, I'm no mathematician ;) These things seem to be used as pseudo-random number generators or
noise generators (see the TIA sound generator, ditto for the GBA) more than anything else.

A polynomial counter (actually a form of "Linear Feedback Shift Register") consists of a shift register, as the name suggests, with some sort of feedback logic - in this case a single two-input XNOR gate obfuscated in NOR logic. They have the property that they will step through up to (2^n)-1 unique states when optimally wired up, from any starting state except for the illegal state (and of course it's possible to power-up in the illegal state =) so for a 6-bit shift register there can be at most 63 valid states.

The TIA uses the same polynomial counter circuit for all of its horizontal counters - there is a HSync counter, two Player Position and two Missile Position counters, and the Ball Position counter. The sound generator has a more complex design involving another polynomial counter or two - I haven't delved into the workings of this one yet.

Beside each counter there is a two-phase clock generator. This takes the incoming 3.58 MHz colour clock (CLK) and divides by 4 using a couple of flip-flops. Two AND gates are then used to generate two independent clock signals thusly:



You'll need a thingo, fixed-spacing font, to make sense of that.
The two clock lines are used to perform a two-step increment
of the counter, as well as being used independently to move
data through the supporting clocked logic.

This concept seems to come up a -lot- in the TIA, I think it's
some sort of Zen NMOS thing, it seems to go hand and hand with
storing data in back of inverters all over the place (a * is used
in the TIA schematics to denote this), and building bit-shifting
chains into your data storage so you don't need addressing ;p
If you've ever wondered why the Playfield bit order is so obscure,
now you know.

Each counter has a wired-AND counter state decode matrix (woo..)
connected in parallel with the shift register. In every case,
the top line of the decoder on the schematics checks for '111111'
and forces a Reset if it is encountered. This is to prevent the
counter getting stuck in the illegal state when it powers up as
mentioned earlier.

Also in every case, the next decoder line is the 'wrap-around'
value - when this state comes up, the counter does a self-reset
to 000000 on the next phase-2 clock, and usually does something
useful like generating a START signal for graphics output.

The rest of the counter decodes depend entirely on which counter
we're looking at, set let's get into 'em.

++

+++ Horizontal Sync Counter

The HSync counter counts from 0 to 56 once for every TV scan-line
before wrapping around, a period of 57 counts at 1/4 CLK (57*4=228
CLK). The counter decodes shown below provide all the horizontal
timing for the control lines used to construct a valid TV signal.

This table shows the elapsed number of CLK, CPU cycles, Playfield
(PF) bits and Playfield pixels at the start of each counter state

(ie when the counter changes to this state on the rising edge of the H@2 clock). The decoded control lines are usually clocked into other logic blocks during the next H@1-H@2 cycle (within 4 CLK).

Value HCount CLK CPU PF Pixel Control

000000 0 0 0
100000 1 4 1.3
110000 2 8 2.6
111000 3 12 4
111100 4 16 5.3 Set H-SYNC [SHS]
111110 5 20 6.6
011111 6 24 8
101111 7 28 9.3
110111 8 32 10.6 Reset H-SYNC [RHS]
111011 9 36 12
111101 10 40 13.3
011110 11 44 14.6
001111 12 48 16 ColourBurst [RCB]
100111 13 52 17.3
110011 14 56 18.6
111001 15 60 20
011100 16 64 21.3 Reset H-BLANK [RHB]
101110 17 68 22.6 0 0
010111 18 72 24 1 4 Late RHB [LRHB]
101011 19 76 25.3 2 8
110101 20 80 26.6 3 12
011010 21 84 28 4 16
001101 22 88 29.3 5 20
000110 23 92 30.6 6 24
000011 24 96 32 7 28
100001 25 100 33.3 8 32
010000 26 104 34.6 9 36
101000 27 108 36 10 40
110100 28 112 37.3 11 44
111010 29 116 38.6 12 48
011101 30 120 40 13 52
001110 31 124 41.3 14 56
000111 32 128 42.6 15 60
100011 33 132 44 16 64
110001 34 136 45.3 17 68
011000 35 140 46.6 18 72
101100 36 144 48 19 76 Center [CNT]
110110 37 148 49.3 20 80

011011 38 152 50.6 21 84
101101 39 156 52 22 88
010110 40 160 53.3 23 92
001011 41 164 54.6 24 96
100101 42 168 56 25 100
010010 43 172 57.3 26 104
001001 44 176 58.6 27 108
000100 45 180 60 28 112
100010 46 184 61.3 29 116
010001 47 188 62.6 30 120
001000 48 192 64 31 124
100100 49 196 65.3 32 128
110010 50 200 66.6 33 132
011001 51 204 68 34 136
001100 52 208 69.3 35 140
100110 53 212 70.6 36 144
010011 54 216 72 37 148
101001 55 220 73.3 38 152
010100 56 224 74.6 39 156 RESET, HBLANK [SHB]
101010 57 (228) (76) (40) (160) (already at 000000)
010101 58 232 - - -
001010 59 236 - - -
000101 60 240 - - -
000010 61 244 - - -
000001 62 248 - - -
000000 0 0 - - - (cycle)
111111 - - - - - ERROR (Reset to 000000)

Key:
SHS Turn on the TV HSYNC signal to start Horizontal flyback.
RHS Turn off the HSYNC signal, delayed 4 CLK.
RCB Reset Colour Burst, delayed 4 CLK latching [CB].
RHB Reset HBlank (enable output), delayed 4 CLK latching [HB].
LRHB Late RHB, used instead of RHB when [HMOVE] latch is set.
CNT Center screen, start copy/reflect PF, delayed 4 CLK for [CNTD].
SHB Start HBlank (disable output), Reset HCount to 000000.

The HSync counter resets itself after 57 counts; the decode on HCount=56 performs a reset to 000000 delayed by 4 CLK, so HCount=57 becomes HCount=0. This gives a period of 57 counts or 228 CLK.

Playfield pixels start on the [RHB] control line at CLK=64, but the first visible pixel won't appear until CLK=68 due to the

clocking on its output. The [CNT] control line either starts the Playfield again as normal, or starts a reverse-shifted copy when reflect-playfield [REF] is enabled.

RSYNC resets the two-phase clock for the HSync counter to the H@1 rising edge when strobed. It looks like this could be used to move the HSync counter into phase with the CPU on any cycle (although there is some auto-synchronisation between the two-phase clock and the div-by-3 counter for the CPU clock, I haven't looked into this yet.) A full H@1-H@2 cycle after RSYNC is strobed, the HSync counter is also reset to 000000 and HBlank is turned on. This one requires more investigation.

++

+++ Player 0 and Player 1 Horizontal Position Counters

There are two independent Player Horizontal Position Counters, one each for player 0 and player 1. The counters are identical; only one is drawn in the schematics. This section describes only the player 0 counter.

The player position counter controls the position of the player graphics object (P0) on each scanline. The player counter counts from 0 to 39 and then wraps around, giving a period of 40 counts at 1/4 CLK (160 CLK) - also the number of visible pixels on a scanline.

This table shows the elapsed number of CLK and CPU cycles at the beginning of each counter state (the CPU column isn't particularly relevant). Each START decode is delayed by 4 CLK in decoding, plus a further 1 CLK to latch the STARTat the graphics scan counter. The START decodes are ANDed with flags from the NUSIZ register before being latched, to determine whether to draw that copy. Actual graphics output is shown in parentheses for non-stretched copies of the player.

Value PCount CPU CLK Event

000000 0 0 0 (draw -012)
100000 1 1.3 4 (draw 3456)
110000 2 2.6 8 (draw 7---)
111000 3 4 12 START DRAWING (NUSIZ=001,011)

111100 4 5.3 16 (draw -012)
111110 5 6.6 20 (draw 3456)
011111 6 8 24 (draw 7---)
101111 7 9.3 28 START DRAWING (NUSIZ=011,010,110)
110111 8 10.6 32 (draw -012)
111011 9 12 36 (draw 3456)
111101 10 13.3 40 (draw 7---)
011110 11 14.6 44
001111 12 16 48
100111 13 17.3 52
110011 14 18.6 56
111001 15 20 60 START DRAWING (NUSIZ=100,110)
011100 16 21.3 64 (draw -012)
101110 17 22.6 68 (draw 3456)
010111 18 24 72 (draw 7---)
101011 19 25.3 76
110101 20 26.6 80
011010 21 28 84
001101 22 29.3 88
000110 23 30.6 92
000011 24 32 96
100001 25 33.3 100
010000 26 34.6 104
101000 27 36 108
110100 28 37.3 112
111010 29 38.6 116
011101 30 40 120
001110 31 41.3 124
000111 32 42.6 128
100011 33 44 132
110001 34 45.3 136
011000 35 46.6 140
101100 36 48 144
110110 37 49.3 148
011011 38 50.6 152
101101 39 52 156 RESET, START DRAWING (always)
010110 40 53.3 160 (already at 000000)
001011 41 54.6
100101 42 56
010010 43 57.3
001001 44 58.6
000100 45 60
100010 46 61.3
010001 47 62.6

001000 48 64
100100 49 65.3
110010 50 66.6
011001 51 68
001100 52 69.3
100110 53 70.6
010011 54 72
101001 55 73.3
010100 56 74.6
101010 57 76
010101 58 -
001010 59 -
000101 60 -
000010 61 -
000001 62 -
000000 0 - (cycle)
111111 - - ERROR (Reset to 000000)

The graphics output for players contains some extra clocking logic not present for the Playfield or other screen objects. It takes 1 additional CLK to latch the player START signal. The rest of the clocking logic is in common with the other grahpics objects; therefore we can say that player grahpics are delayed by 1 CLK (this is why the leftmost possible start position for a RESP0 is pixel 1, not pixel 0. You can HMOVE the player further left though, if necessary.)

The most important thing to note about the player counter is that it only receives CLK signals during the visible part of each scanline, when HBlank is off; exactly 160 CLK per scanline (except during HMOVE). During the other 68 CLK per line, the counter lies dormant on the exact 1/4 phase it was up to. The [MOTCK] (motion clock?) line supplies the CLK signals for all movable graphics objects during the visible part of the scanline. It is an inverted (out of phase) CLK signal.

This arrangement means that resetting the player counter on any visible pixel will cause the main copy of the player to appear at that same pixel position on the next and subsequent scanlines. There are 5 CLK worth of clocking/latching to take into account, so the actual position ends up 5 pixels to the right of the reset pixel (approx. 9 pixels after the start of STA RESP0).

For better or worse, the manual 'reset' signal (RESP0) does not

generate a START signal for graphics output. This means that you must always do a 'reset' then wait for the counter to wrap around (160 CLK later) before the main copy of the player will appear. However, if you have any of the 'close', 'medium' or 'far' copies of the player enabled in NUSIZ, these will be drawn on the current and subsequent scanlines as the appropriate decodes are reached and generate their START signals.

++

+++ Player 0 and Player 1 Graphics Scan Counters

The Player Graphics Scan Counters are 3-bit binary ripple counters attached to the player objects, used to determine which pixel of the player is currently being drawn by generating a 3-bit source pixel address. These are the only binary ripple counters in the TIA.

The Scan Counters are never reset, so once the counter receives the Start signal it will count fully from 0 to 7. Counting is only performed during the visible part of the scanline since it is driven by the [MOTCK] line used to advance the Player Position Counter. This gives rise to "sprite wrapping" whereby a player positioned so it ends past the righthand side of the screen will finish drawing at the beginning of the next scanline. Note that a HMOVE can gobble up the wrapped player graphics - see below.

The count frequency is determined by the NUSIZ register for that player; this is used to selectively mask off the clock signals to the Graphics Scan Counter. Depending on the player stretch mode, one clock signal is allowed through every 1, 2 or 4 graphics CLK. The stretched modes are derived from the two-phase clock; the H@2 phase allows 1 in 4 CLK through (4x stretch), both phases ORed together allow 1 in 2 CLK through (2x stretch).

The NUSIZ register can be changed at any time in order to alter the counting frequency, since it is read every graphics CLK. This should allow possible player graphics warp effects etc.

Player Reflect bit - this is read every time a pixel is generated, and used to conditionally invert the bits of the source pixel address. This has the effect of flipping the player image drawn.

This flag could potentially be changed during the rendering of the player, for example this might be used to draw bits 01233210.

Player graphics registers - there are four 8-bit registers in the TIA for storing Player graphics, two for each player. Only two of these are ever directly accessible; these are labelled the "new" player graphics registers on the schematics. Unless the Player Vertical Delay (VDELPn) is set, the "new" registers are always drawn.

Writes to GRP0 always modify the "new" P0 value, and the contents of the "new" P0 are copied into "old" P0 whenever GRP1 is written. (Likewise, writes to GRP1 always modify the "new" P1 value, and the contents of the "new" P1 are copied into "old" P1 whenever GRP0 is written). It is safe to modify GRPn at any time, with immediate effect.

Vertical Delay bit - this is also read every time a pixel is generated and used to select which of the "new" (0) or "old" (1) Player Graphics registers is used to generate the pixel. (ie the pixel is retrieved from both registers in parallel, and this flag used to choose between them at the graphics output). It is safe to modify VDELPn at any time, with immediate effect.

++

+++ Missile 0 and Missile 1 Horizontal Position Counters

There are also two individual Horizontal Position Counters for missile 0 and missile 1. The counters are independent and identical.

These counters use exactly the same counter decodes as the players, but without the extra 1 CLK delay to start writing out graphics.

Missiles use the same control lines as the player from the NUISZ register to determine the number of copies drawn, although they ignore the player scaling options (you'll just get a single copy for the scaled player modes).

Missile width is implemented in the same way as the ball width; it appears to be exactly the same gate arrangement (see below).

The Missile-to-player reset is implemented by resetting the M0

counter when the P0 graphics scan counter is at %100 (in the middle of drawing the player graphics) AND the main copy of P0 is being drawn (ie the missile counter will not be reset when a subsequent copy is drawn, if any). This second condition is generated from a latch outputting [FSTOB] that is reset when the P0 counter wraps around, and set when the START signal is decoded for a 'close', 'medium' or 'far' copy of P0.

++

+++ Ball Horizontal Position Counter

The ball position counter controls the position of the ball graphics object (BL) on each scanline. The ball counter counts from 0 to 39 and then wraps around, giving a period of 40 counts at 1/4 CLK (160 CLK).

Ball width is given by combining clock signals of different widths based on the state of the two size bits (the gates form an AND -> OR -> AND -> OR -> out arrangement, with a hanger-on AND gate). See notes later for all the messy details ;p

It seems a shame to have a whole polynomial counter for the ball, and
no special effects aside from its size - except for one small detail.

If you look closely at the START signal for the ball, unlike all the other position counters - the ball reset RESBL does send a START signal for graphics output! This makes the ball incredibly useful since you can trigger it as many times as you like across the same scanline and it will start drawing immediately each time :)

So it's good for cutting holes in things, drawing background details, clipping the edges off things, etc. It can even be used to draw simple sprites, or used as the background colour (because it's behind everything else) for a two-colour sprite.

Actually on my 2600jr (long rainbow), setting the ball size to 8 pixels results in solid colour when it's reset every 9 pixels (this might just be colour bleeding, I'm not sure).

Value BCount CPU CLK Event

000000 0 0 0 (draw 0123)
100000 1 1.3 4 (draw 4567)
110000 2 2.6 8
111000 3 4 12
111100 4 5.3 16
111110 5 6.6 20
011111 6 8 24
101111 7 9.3 28
110111 8 10.6 32
111011 9 12 36
111101 10 13.3 40
011110 11 14.6 44
001111 12 16 48
100111 13 17.3 52
110011 14 18.6 56
111001 15 20 60
011100 16 21.3 64
101110 17 22.6 68
010111 18 24 72
101011 19 25.3 76
110101 20 26.6 80
011010 21 28 84
001101 22 29.3 88
000110 23 30.6 92
000011 24 32 96
100001 25 33.3 100
010000 26 34.6 104
101000 27 36 108
110100 28 37.3 112
111010 29 38.6 116
011101 30 40 120
001110 31 41.3 124
000111 32 42.6 128
100011 33 44 132
110001 34 45.3 136
011000 35 46.6 140
101100 36 48 144
110110 37 49.3 148
011011 38 50.6 152
101101 39 52 156 RESET, START DRAWING
010110 40 53.3
001011 41 54.6
100101 42 56
010010 43 57.3

001001 44 58.6
000100 45 60
100010 46 61.3
010001 47 62.6
001000 48 64
100100 49 65.3
110010 50 66.6
011001 51 68
001100 52 69.3
100110 53 70.6
010011 54 72
101001 55 73.3
010100 56 74.6
101010 57 76
010101 58 -
001010 59 -
000101 60 -
000010 61 -
000001 62 -
000000 0 - (cycle)
111111 - - ERROR (Reset to 000000)

Vertical Delay bit - the VDELBL control bit works in the same manner as the player VDEL bits; the state of VDELBL is used every CLK to determine which of the "new" (0) or "old" (1) ENABL values to use at the graphics output. Writes to ENABL always modify the "new" value, and whenever GRP1 is written the "new" value is copied into the "old". It is safe to modify VDELBL and ENABL at any time, with immediate effects.

++

+++ Using the Horizontal Position Counters

The documented way to use a player position counter is to reset it with RESPn on any CPU cycle divisible by 5 during the visible scanline (5 is a convenient number for DEX-BNE loops), set up HMPn to adjust the position by +7 (left) to -8 (right) pixels, and hit HMOVE immediately after the next WSYNC. Then configure NUSIZn for the number and spacing of copies required, and let the hardware go about its business. Once this is set up, you can just change the grpahics in GRPn every scanline to get one, two or three copies at fixed spacing.

In fact the hardware has hard-wired requirements for almost none of the above =) The fixed spacing between copies is hard-wired and HMOVE is largely not negotiable, but the rest is complete tosh.

The TIA renders each movable graphics object according to independent position counters running at 1/4 CLK with a period of 40 increments, and synchronised to the last RESPn/RESMn/RESBL strobe. Each and every time a counter wraps around, the 'main' copy of the object starts to draw. Since it takes 4 CLK to reset the counter to zero and 4 CLK to increment the counter, the image can be expected to appear after exactly 40 full counts, or 160 CLK.

The counters are normally only running during the 'visible' part of a scanline, unless you're doing a HMOVE. Since the scanline has 160 visible pixels, this yields the documented behavior that a RESPn/etc sets the position for the next scanline. It's out by 5 pixels when you set it, but who's counting?

Due to extra clocking logic for Player graphics output, the first player pixel won't appear until 1 CLK later than for any other grahpics object once rendering 'starts'. See the HSync/Player Counter info above for an explanation of this.

During the horizontal blank (see the Horizontal Counter info above) the Player, Missile and Ball counters stop receiving CLK signals, so they pause on the exact 1/4 CLK they're up to and resume where they left off at the first visible pixel on the next scanline. This gives rise to the 'wrap around' effect, to the point of splitting a copy of the player image in half because it happened to start too near the right edge of the screen ;)

The object counters are running at the same 1/4 CLK rate as the HSync counter, but you can set them out of phase with the HSync counter (and therefore the Playfield) by resetting any of them on a CPU cycle that isn't divisible by 4. (If this were not the case, there would only be 40 possible positions along the scanline and we could all go home early). You can also use the HMOVE command, which is described below.

++

++++++++++++++++++++++++++
+++ Playing with the HMOVE registers

In principle the operation of HMOVE is quite straight-forward; if a HMOVE is initiated immediately after HBlank starts, which is the case when HMOVE is used as documented, the [HMOVE] signal is latched and used to delay the end of the HBlank by exactly 8 CLK, or two counts of the HSync Counter. This is achieved in the TIA by resetting the HB (HBlank) latch on the [LRHB] (Late Reset H-Blank) counter decode rather than the normal [RHB] (Reset H-Blank) decode.

The extra HBlank time shifts everything except the Playfield right by 8 pixels, because the position counters will now resume counting 8 CLK later than they would have without the HMOVE. This is also the source of the HMOVE 'comb' effect; the extended HBlank hides the normal playfield output for the first 8 pixels of the line.

In order to move less than 8 pixels right the TIA performs 'clock stuffing' on the Player, Missile and Ball position counters, whereby a number of clock pulses between 0 and 15 are sent to the counters during HMOVE. Each extra clock pulse eats up 1/4 count in the object's horizontal position counter, and thereby moves the object left one pixel. This must be done during HBlank because it is sending these extra clock pulses down the same clock lines that usually receive [MOTCK] pulses during the visible part of the scanline.

The Stella Programmer's Guide states that "the motion registers should not be modified for at least 24 machine cycles after an HMOVE command". This is indeed for internal hardware considerations, although perhaps not entirely mysterious. After several attempts, I finally got my head around the heavily obfuscated logic in the schematics. It turns out to be fairly simple, and quite elegant :)

The HMOVE values set by the programmer are stored in a matrix of 4-bit data latches with built-in comparators - each latch effectively contains a wired-XOR gate, and the 4 latches for a given HMxx register are arranged in a wired-NOR formation to give a 4-bit comparator.

Beside the matrix of HMxx latches is a 4-bit binary ripple

counter. It begins at 15 and decrements down to zero during the HMOVE at a rate of 1 decrement every 4 CLK (it's built from 2-phase clocked logic). The counter is wired in parallel to the comparators in all 5 HMxx registers.

At the beginning of the HMOVE, a latch is set for each movable object to indicate that it requires more motion to the left. When the comparator for a given object detects that none of the 4 bits match the bits in the counter state, it clears this latch (a clever exercise in reverse logic!) Until this time, the output of the latch is sent through to the movable object once every 4 CLK (on every H@1 signal from the HSync two-phase clock) as an extra "stuffed" clock signal.

Since one extra CLK pulse is sent every 4 CLK, this takes at most 4*16=64 CLK (including counter reset at the end), or 64/3=21 CPU cycles. It takes 3 CLK after the HMOVE command is received to decode the [SEC] signal (at most 6 CLK depending on the timing of STA HMOVE) and a further 4 CLK to set the "more movement required" latches. So we need to wait at least 71/3=23.66 CPU cycles before the HMOVE operation is complete. For a normal HMOVE after WSYNC, it might be safe by cycle 23 (this has not been tested).

The first compare (against 15) will be sampled 15 CLK after STA HMOVE begins and every 4 CLK thereafter. The first counter decrement will happen at CLK 17, and every 4 CLK thereafter.

You may have noticed that the above discussion ignores the fact that HMxx values are specified in the range +7 to -8. In an odd twist, this was done purely for the convenience of the programmer! The comparator for D7 in each HMxx latch is wired up in reverse, costing nothing in silicon and effectively inverting this bit so that the value can be treated as a simple 0-15 count for movement left. It might be easier to think of this as having D7 inverted when it is stored in the first place.

In theory then the side effects of modifying the HMxx registers during HMOVE should be quite straight-forward. If the internal counter has not yet reached the value in HMxx, a new value greater than this (in 0-15 terms) will work normally. Conversely, if the counter has already reached the value in HMxx, new values will have no effect because the latch will have been cleared.

Much more interesting is this: if the counter has not yet reached the value in HMxx (or has reached it but not yet commited the comparison) and a value with at least one bit in common with all remaining internal counter states is written (zeros or ones), the stopping condition will never be reached and the object will be moved a full 15 pixels left. In addition to this, the HMOVE will complete without clearing the "more movement required" latch, and so will continue to send an additional clock signal every 4 CLK (during visible and non-visible parts of the scanline) until another HMOVE operation clears the latch. The HMCLR command does not reset these latches.

The Cosmic Ark stars effect achieved this by writing the value $60 to HMM0, 21 cycles after HMOVE starts. See this message in the archives:
http://www.biglist.com/lists/stella/archives/199705/msg00024.html

Following is how I believe it works: at 21 cycles in, the internal counter has just decremented to %0000 and is about to test this against the HMxx registers (2 CLK from now, if my timings are correct). If we flip the top bit of $60 as described above, we have the binary pattern %1110. This pattern has at least one bit in common with the final remaining state (the bottom zero bit), and also has bits in common with the default counter state %1111 which will arise when the counter resets. This means the compare will pass now and forever more :) For this to work, I expect that they must have set HMM0 to $70 before using the trick (binary %0111, or %1111 with the bit flipped), but after a cursory glance at Thomas' commented Cosmic Ark code I haven't found this.

Looking at the archives relating to Cosmic Arc and Rabbit Transit tricks, I also notice that a HMCLR 20 cycles in has the same effect. In this case it will be resetting HMxx to %1000 (bit-flipped) which also obeys the rules for bypassing the stopping condition.

Also of note, the HMOVE latch used to extend the HBlank time is cleared when the HSync Counter wraps around. This fact is exploited by the trick that invloves hitting HMOVE on the 74th CPU cycle of the scanline; the CLK stuffing will still take place during the HBlank and the HSYNC latch will be set just before the counter wraps around. It will then be cleared again immediately (and therefore ignored) when the counter wraps,

preventing the HMOVE comb effect. Since the extended HBlank is needed to move all objects right 8 pixels, this has the limitation that objects can only be moved left, and the normal HMOVE numbering no longer applies. Instead the HMOVE value is interpreted as (8 + value) pixels to the left, ie:

-8 = 0 -4 = 4 0 = 8 4 = 12
-7 = 1 -3 = 5 1 = 9 5 = 13
-6 = 2 -2 = 6 2 = 10 6 = 14
-5 = 3 -1 = 7 3 = 11 7 = 15

This means that all objects will be moved 8 pixels left unless you set their HMxx value to -8 for zero movement.

I've recently found a post in the Stella mailing list archives that gave these results by exhaustive testing, posted by Brad Mott: http://www.biglist.com/lists/stella/archives/199804/msg00198.html

++
+++ Graphics Scan Counters during HMOVE

Since the Graphics Scan Counters are never reset, player graphics output can wrap around as mentioned above.

A HMOVE 8 pixels right (-8 << 4), has no effect on the scan counter since it will perform no "clock stuffing" of the player counters for that player (the extended HBlank time moves everything right 8 pixels).

Any other HMOVE value will gobble up at least one pixel, or more proportional to the HMOVE value. Since a HMOVE value really represents a count from 0 (for -8) to 15 (for +7) with the top bit inverted, this is the number of player pixels that will be gobbled up by the HMOVE.

This means that a HMOVE of 0 will gobble up all remaining wrapped output for the non-stretched player modes, since it sends 8 extra clocks to the player. (Note that this is only true if HMOVE was actually strobed for the scanline, otherwise the configured HMxx registers never have any effect). For the stretched player modes there could be some output left - it takes 16 stuffed clocks to eat up a full

2X player, and 32 clocks to eat up a full 4X player.

++

+++ HMOVE during the visible scanline.

I mentioned above that HMOVE sends extra clock pulses down the same clock lines that are usually used during the visible part of the scanline. In theory this means that performing a HMOVE during the visible part of the scanline should have no effect. However, looking at how the various clock signals interact, I suspect it is possible. I did some preliminary experiments (on a 2600 Jr) at some point, and I seem to remember having some success.

In this case the extra HMOVE clock pulses act to perform 'plugging' instead of the normal 'stuffing'; by this I mean that the extra pulses plug up the gaps in the normal [MOTCK] pulses, preventing them from counting as clock pulses. This only works because the extra HMOVE pulses are derived from the two-phase clock on the HSync counter, which is itself derived from CLK (the TIA colour clock input), whereas [MOTCK] is an inverted CLK signal - so they are more or less precisely out of phase :)

I'm not sure how universal (or reliable!) this might turn out to be, but I haven't seen it mentioned before. Also of note, this technique can only be used to effect a move to the right, at a rate of 1 pixel every 4 CLK (since this is the rate that HMOVE generates the extra clock pulses).

++

+++ The Re-trigger Trick, and all that jazz

I've read some theories suggesting that re-triggering is a hack, possibly dependent on chip revision, where you trick the TIA into rendering more than three copies by hitting RESP0/RESP1 during the rendering of a 'legitimate' copy, or some other method to confuse the poor chip. Through extensive coffee consumption, I have determined that this is not the case. Perhaps peering at the TIA schematics for countless

hours on end, until I fell asleep (two days in a row), may have helped also.

The behaviour of the TIA positioning registers is quite predictable and completely independent from its graphics output logic, as documented above. What remains are issues involving the timing of RESPn commands, given that the TIA counts things at 1/4 clock and the CPU runs at 1/3 CLK =)

Following is a table of the cycle decodes for the Player counters, starting from CLK=0 when the counter resets. This is an excerpt from the Player Counter table listed elsewhere in the document (I recomment you go have a look, the spacing between events should look oddly familiar ;)

Value PCount CPU CLK Event

111000 3 4 12 START DRAWING (NUSIZ=001,011) Close
101111 7 9.3 28 START DRAWING (011,010,110) Medium
111001 15 20 60 START DRAWING (100,110) Far
101101 39 52 156 RESET, START DRAWING (always) Main

The columns from the left are: the polynomial counter state, (see notes above), the decimal value that the player counter is up to, the number of CPU cycles since the counter reset, and the number of CLKs elapsed since the counter reset.

You'll notice I'm now talking about everything relative to RESPn on the current scanline, rather than the beginning of the scanline. This is because this is all that matters.
You should understand the following point:

If you hit RESPn at least twice on every scanline,
you will never see the 'main' copy of that player,
ever, on any scanline.

This is because the counter will always be reset before it manages to complete a full 40 counts (160 CLK), and so the 'main' copy will never start drawing.

This is tricky to test, especially if you don't reset a few things when you stop (eg, for VSync) - whenever you stop hiting RESPn, you will start to get the normal output on the next and subsequent scanlines, including the 'main' copy.

The very top visible scanline is a perfectly valid 'subsequent scanline' after the very bottom visible scanline, once you get past the first frame ;)

If you've set up NUSIZn for 3 copies close (011), you'll be getting four copies on every scanline on which you hit RESPn twice, as long as they are far enough apart. This works because it doesn't take a counter wrap-around to get to the 'close' and 'medium' copies as shown in the table above. They will appear 4+12+1=17 and 4+28+1=33 pixels after each RESPn CLK arrives in the TIA (it takes 4 extra CLK to reset the counter, and 1 extra CLK to start the graphics output).

It's important to note, that as long as the second RESPn on the line causes a reset after the 'start' signal has been generated for the 'medium' copy of the first RESPn, you will get four copies regardless of how far apart the RESPn hits are. If you do the second RESPn too soon you'll end up with only three copies - the 'close' from the first RESPn, followed by the the 'close' and the 'medium' from the second RESPn. If you do the second RESPn before the first 'close' copy, you'll only end up with the 'close' and 'medium' from the second RESPn.

From this it follows that if you set NUSIZ0 to 011, hit RESPn and wait until the 'medium' copy has started, then change NUSIZ0 to 100 or 110, you will get all of 'close', 'medium' and 'far'.

++

+++ Re-triggering after exactly 18, 33, 66 or 162 cycles

These are special cases only because resetting a Position Counter (RSPn, RESMn, RESBL) also resets the two-phase clock attached to it, and this in turn affects the clocked logic on the output of the counter decodes.

For the player counters, this affects the four decodes that produce the Start signal for copies of the player graphics. These are generated 12, 28, 60, and 160 CLK after the Position Counter has been reset, in order to trigger the 'close', 'medium', 'far' and 'main' copies.

These decodes pass through a block of logic that requires a full cycle of the two-phase clock (hence the normal 4 CLK delay before graphics output common to all movable graphics objects). If the Position Counter and therefore the two-phase clock are reset during this decoding process, the Start signal will either be lost or delayed up to 3 CLK depending on exact timing.

This effect is most evident when attempting to re-trigger the player graphics over and over again. For example, examine this retriggering technique:

```
STA RESP0 ;3 reset P0, call this 0 CLK.
CMP $EA ;3 nop
STA RESP0 ;3 reset P0 again, after 18 CLK.
CMP $EA ;3 nop
STA RESP0 ;3 reset P0 again, after 18 CLK.
```

The visible result of this will be a 'close' copy of P0 shifted right by two pixels from the expected position, followed by a second 'close' copy shifted right by two pixels, and finally a third 'close' copy, not shifted right. There will be an 18 pixel gap between the first two copies of P0, and only a 16 pixel gap before the third copy.

In order to fix up the spacing of the final copy, it is necessary to trigger P0 yet again exactly 18 CLK later, but clear GRP0 in the mean time so nothing is drawn.

If the retriggering will be continuing onto the next line there is no need to do this; just ensure that the first re-trigger on the next line happens 18 visible pixels after the last RESP0 on the previous line (ie 18 CLK later, minus HBlank time).

++

+++ Notes on the Ball/Missile width enclockifier

Just to reiterate, ball width is given by combining clock signals of different widths based on the state of the two size bits (the gates form an AND -> OR -> AND -> OR -> out arrangement, with a hanger-on AND gate).

The Enable (output) signal is built in two halves, arranged back-

to-back at the final OR gate.

The first half comes from one of three sources combined through the earlier OR gate and then AND-ed with the Start signal:

(1) If D4 and D5 are both clear, one of the two-phase clock signals (active 1 in 4 colour CLK) yields a single pixel of output.
(2) If D4 is set, a line active 2 in every 4 colour CLK is borrowed from the two-phase clock generator (this yields 2 pixels).
(3) Finally D5 itself is used directly - the Start signal is active for 4 CLK so this generates 4 pixels.

The second half is added if both D4 and D5 are set; a delayed copy of the Start signal (4 colour CLK wide again) is OR-ed into the Enable signal at the final OR gate.

I hope someone had as much fun building this little circuit as I had pulling it apart again ;p

++
+++ CPU Clock to Player Pixel Table

The Player Position Counter can be reset to zero (with RESP0/1) on any CPU cycle as shown below, and copies will appear at the pixel positions listed for 'close', 'medium' and/or 'far' depending on the flags in NUSIZ; 1, 2, 3 or (if you change NUSIZ at the right time) 4 copies at hard-wired positions after the reset. If the counter is allowed to wrap around, the 'main' copy will appear on the next line.

Resetting the counter takes 4 CLK, decoding the 'start drawing' signal takes 4 CLK, latching the 'start' takes a further 1 CLK giving a total 9 CLK delay after a RESP0/1. Since the playfield takes 4 CLK to start drawing the player is visibly delayed by exactly 5 CLK - hence the magic '5' :)

NOTE: The player counter can be safely reset 18 CLK after the previous
reset and the previous copy will still be drawn. BUT the 'start' signal for the previous copy will be delayed a further 2 CLK due to the 2-phase clock being reset before the 'start' signal has been clocked through to the 'start' latch.

CPU CLK Pixel Main Close Medium Far PF

0 0 - 1 17 33 65 -
...
22 66 - 1 17 33 65 -
22.6 ----------
23 69 1 6 22 38 70 0.25
24 72 4 9 25 41 73 1
25 75 7 12 28 44 76 1.75
26 78 10 15 31 47 79 2.5
27 81 13 18 34 50 82 3.25
28 84 16 21 37 53 85 3
29 87 19 24 40 56 88
30 90 22 27 43 59 91
31 93 25 30 46 62 94
32 96 28 33 49 65 97
33 99 31 36 52 68 100
34 102 34 39 55 71 103
35 105 37 42 58 74 106
36 108 40 45 61 77 109
37 111 43 48 64 80 112
38 114 46 51 67 83 115
39 117 49 54 70 86 118
40 120 52 57 73 89 121
41 123 55 60 76 92 124
42 126 58 63 79 95 127
43 129 61 66 82 98 130
44 132 64 69 85 101 133
45 135 67 72 88 104 136
46 138 70 75 91 107 139
47 141 73 78 94 110 142
48 144 76 81 97 113 145
49 147 79 84 100 116 148
50 150 82 87 103 119 151
51 153 85 90 106 122 154
52 156 88 93 109 125 157
53 159 91 96 112 128 0
54 162 94 99 115 131 3
55 165 97 102 118 134 6
56 168 100 105 121 137 9
57 171 103 108 124 140 12
58 174 106 111 127 143 15
59 177 109 114 130 146 18

60 180 112 117 133 149 21
61 183 115 120 136 152 24
62 186 118 123 139 155 27
63 189 121 126 142 158 30
64 192 124 129 145 1 33
65 195 127 132 148 4 36
66 198 130 135 151 7 39
67 201 133 138 154 10 42
68 204 136 141 157 13 45
69 207 139 144 0 16 48
70 210 142 147 3 19 51
71 213 145 150 6 22 54
72 216 148 153 9 25 57
73 219 151 156 12 28 60
74 222 154 159 15 31 63
75 225 157 2 18 34 66
76 228 0 5 21 37 69
---------- Start HBLANK

Also note that hitting RESP0 before HBLANK has finished will reset the counter immediately, but it will only start counting again when HBLANK goes off. Due to output clocking, this will produce player graphics at playfield pixel 1.

++

+++ The Venerable 6-digit Score Trick

The 6-digit score trick involves putting both players into 3-repeat mode (011 or 110 in NUSIZ0/1) and resetting them such that all the player 2 images are positioned exactly between all the player 1 images, ergo:

P1 P2
v v
1 2 1 2 1 2

Then you need to set the graphics up (GRP0/1) for the first two digits, and write some very precise timing code to wait until the scan-line is just about to start drawing the first copy of P1. While you're waiting, get the rest of the graphics loaded into the registers (A, X, and Y).

At this point you need to start storing all the graphics you've loaded into GRP0 and GRP1 as fast as you can - it will look like this because there's only one way to do it fast enough:

STA GRP0 ; 3
STX GRP1 ; 3
STY GRP0 ; 3
ST? GRP1 ; 3 we've run out of registers!

Notice that each one takes 3 cycles to execute (which is 9 pixels) and makes the change on the -end- of the 3rd cycle. We could use the stack pointer register (S) for the last one and do a TSX, but that would take 5 cycles (that's 15 pixels) which is too long.

To get it working you need to turn on VDELP0/1 (vertical delay) which allows you to set up the first 3 digits in the TIA's graphics registers before the beginning of the scanline, and requires only the 3 remaining registers to hold the last 3 digits.

I've found a post in the Stellar archives that explains this technique in great detail, so I'll stop here.

http://www.biglist.com/lists/stella/archives/199704/msg00137.html

++
+++ Fine Print

Please note that these notes are my own, and are made available without any warranties of any kind. They may include errors, omissions and much that is apocryphal; use at your own risk.

Please let me know if you spot anything that is blatantly wrong and I'll update the document. I'm also happy to answer any questions about this stuff.

Copyright (C) Andrew Towers 2003